PART 4

Track system

Turning a pile of curve objects into a railway the add-on *understands*: accurate, cached geometry; a network that knows which pieces join and where the switches are; banking; and the tools that build and check it all.

4.1 Anatomy of a track piece
4.2 Arc tables done right
4.3 The Track Network
4.4 Turnouts and overlap zones
4.5 Joints and auto-continue
4.6 Banking (cant) tools
4.7 Track Builder and presets
4.8 Track Doctor (validation)

4.1 Anatomy of a track piece

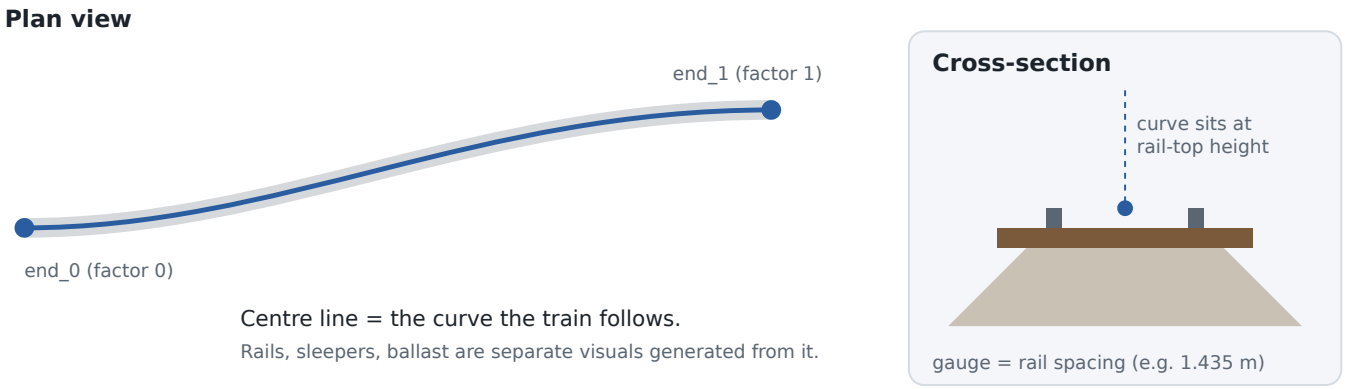


Figure 4.1 Recommended convention: the curve runs along the **centre line at rail-top height**. A vehicle root with origin at rail-top then needs no vertical offset, and pivots land on the rails.

4.2 Arc tables done right

The arc table is the add-on's picture of each curve. Today it is rebuilt from the Bézier formula, ignores cyclic splines, and **is never refreshed after the curve is edited** (audit #2). The fix keeps the same idea and makes it exact and self-maintaining:

1. **Sample the way Blender does**
   Use `mathutils.geometry.interpolate_bezier(knot1, handle1, handle2, knot2, resolution_u + 1)` for each segment. This is the function the first session tried to call with a float. It returns exactly the points Blender's own polyline uses. Include the closing segment when `use_cyclic_u` is on. Transform by `matrix_world`.

2 **Store cumulative length per polyline vertex**
Offsets then map to positions by **interpolating between two vertices**, not snapping to the nearest of 5,000 samples. This is more exact, and the tables are about 20× smaller.

3 **Nearest-point via KD-tree + segment projection**
Build a `mathutils.kdtree.KDTree` of the vertices. For a query, take the few nearest vertices, project onto their adjacent segments, and keep the best. That is O(log n) instead of a 5,000-point loop on every tick.

4 **Key the cache by geometry, not name**
`geo_hash` = hash of point coordinates, handles, tilt, `matrix_world`, `resolution_u` and cyclic flag. Check it cheaply (on `depsgraph_update_post`, only for curves listed in `depsgraph.updates`) and mark dirty on change.



Nothing writes RNA in this path. It only drops Python caches, so it is safe even during a modal transform.

Figure 4.2 Cache invalidation flow.

4.3 The Track Network

Today the add-on discovers relationships *while driving*, by scanning every curve for proximity on every tick. The Track Network does that work **once**, with a **Scan Track** button (and automatically when the geometry changes), and stores the answer as a graph:

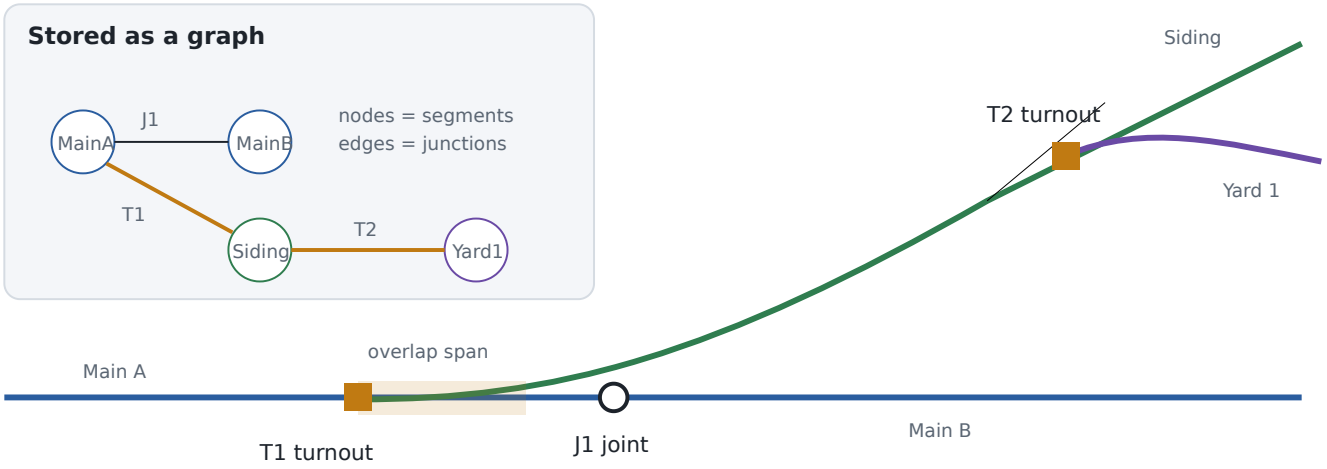


Figure 4.3 A small layout and its network. Each junction records the legs, where on each curve they meet (offset), and whether factor runs the same or opposite way across it.

How Scan Track classifies connections

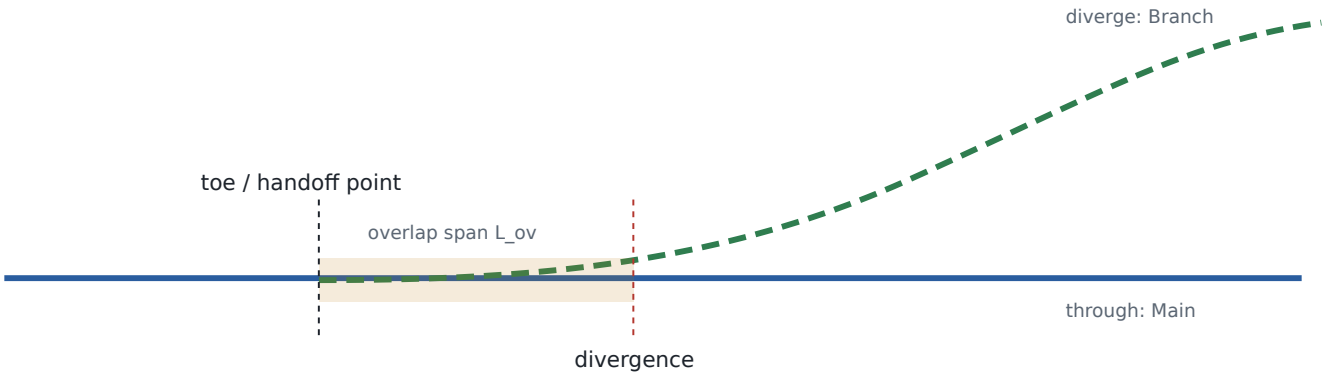
Kind	Detected when…	Stored
Joint	An endpoint of A is within tolerance of an endpoint of B, and the tangents are parallel (either direction).	(A, end), (B, end), direction flip flag
Turnout	An endpoint of B lies on A's *interior*, and B hugs A for a **continuous span** before separating.	A offset range, B offset range, divergence point, span length, which leg is “through”
Crossing / diamond	A and B intersect at an angle, with no shared span.	Informational only: no switching, and a warning if a vehicle could get confused
Dead end	An endpoint with nothing near it.	A buffer stop. Vehicles stop here (factor clamped)

THIS ALSO FIXES V8 §8.5 (YARD FALSE POSITIVE)

The span test in the Turnout row is the “overlap span” fix proposed in v8. Several dead-end sidings that just happen to end near each other fail it, because they don't *hug* each other over a length. They are classified as dead ends, and a train can never jump between them.

4.4 Turnouts and overlap zones

A real turnout has a **toe** (where the diverging route starts), **points** that move, and a **heel/frog** where the routes separate. In curve terms:



Handoff happens at a **single point** where both curves have the same position **and** heading.

Figure 4.4 The anatomy Scan Track looks for. The branch should start **tangent** to the main line at the toe.

How long does the overlap need to be?

Model	Rule
Today (shadow-tracked switching)	L_ov ≥ the vehicle's longest pivot offset, and for trains ≥ the whole consist length, because pivots and wagons clamp to the root's curve (v8 §3.3).
With the Route Plan (Part 5)	L_ov only needs to be **long enough to place the toe accurately** (a few metres). Every point on the train switches when *it* reaches the toe, because the route itself changes curve there. Length stops being a constraint.

DECISION

Should the toe be the **start** of the overlap (natural for turnouts built by duplicating a piece of main line) or an explicit **marker Empty** the user places? Recommendation: auto-detect the start of the overlap, and allow an optional override Empty for odd geometry.

4.5 Joints and auto-continue

A joint is a switch with only one option. In the network model the route **simply continues** across a joint, with no button and no live-switch logic. The requested **“auto-jump to next spline” toggle** becomes a per-junction or scene option:

- **On** (default): the route runs through the joint.
- **Off**: the joint acts as a buffer stop until enabled. This is useful for yard staging, and it can later become a per-vehicle setting (a shunter stops at every joint; an express never does).

4.6 Banking (cant) tools

Banking already works through point tilt. What's missing is **authoring help**. Railways set cant from curve radius and design speed:

```
equilibrium cant angle  θ ≈ atan( v² / (g · R) )        v = design speed (m/s), R = radius (m), g = 9.81
real railways cap cant: ~150 mm on 1.435 m gauge  → θ_max ≈ 6°
```

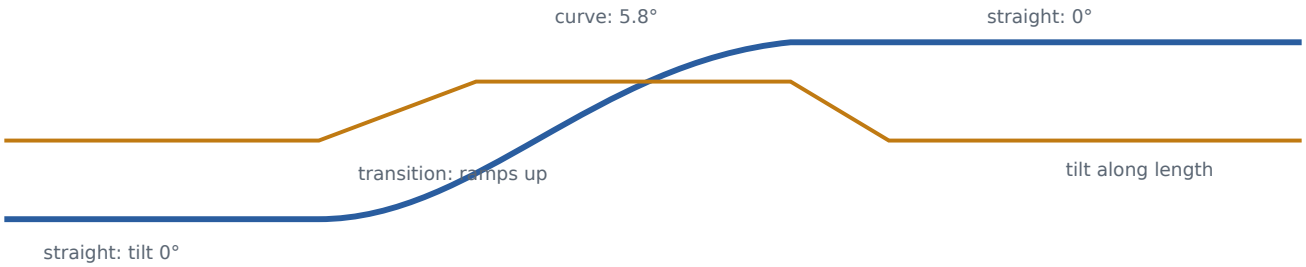


Figure 4.5 The **Auto Cant** operator: measure curvature at each point, apply θ capped at a maximum, and ramp it over transition length so the train rolls in smoothly.

4.7 Track Builder and presets

The legacy “Track It” and “Bank It” buttons build shrinkwrap planes for a rig that no longer exists (audit #13). Replace them with:

Track Builder (Geometry Nodes)

One node-group modifier on the curve: rails (profile swept along the curve), sleepers (instances on points at spacing), ballast. It follows tilt automatically. Parameters: gauge, sleeper spacing, rail profile. It doesn't touch the curve the train rides.

Preset pieces

An asset library of ready track: straights (e.g. 25 m), curves by radius, and **left/right turnouts with a correct toe and overlap already built**. Snap pieces end to end (joints) and Scan Track wires them up. This is the “preset tracks” plan from session 4.

4.8 Track Doctor (validation)

A single button that lists problems, each with a “Select” and, where possible, “Fix” action:

Check	Fix offered
More than one spline in a track curve	Separate into objects
Object scale ≠ 1, or non-uniform	Apply scale
resolution_u below 12	Raise to 12
Twist method not Z-Up	Set Z-Up
Joint gap or tangent mismatch above tolerance	Snap endpoints / align handles
Branch not tangent at the toe	Align first handle to main tangent

Check	Fix offered
Overlap span shorter than the longest vehicle (legacy mode only)	Report which vehicles are affected
Sharp kinks (radius below minimum for vehicles in the scene)	Highlight; suggest smoothing
Direction reversed on a keyed track	Warn before flipping